Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

High-Performance Computing Lab for CSE **2026**

Student: Riccardo Polo Discussed with: Giovanni Guidarini

Report for Project 4

Due date: 27 April 2026, 18:00 (Europe/Zurich)

HPC Lab for CSE 2025 — Submission Instructions

Following these instructions is mandatory: Submissions that do not comply **will not be considered**.

- **Submission Platform**: Assignments must be submitted via SAM-UP.
- **Required Files**: Your submission must include:
  - **Report**: A PDF summarizing your results and observations, based on this LaTeX template.
    - File name: `project_number_lastname_firstname.pdf`
    - You are allowed to discuss all questions with anyone you like; however: (i) your submission must list anyone you discussed problems with and (ii) you must write up your submission independently.
  - **Source Code Archive**: A **compressed tar archive**, named `project_number_lastname_firstname.tar.gz`, containing:
    - All source code files.
    - Build files and scripts (e.g., `Makefile`, batch job scripts, plot scripts, etc). If you modified any provided files, ensure they still work as expected.
    - The full LaTeX source of your report including all needed files to build the report (figures, listings, etc).
  - **Size Limit**: The total submission (PDF + archive) must not exceed **25 MB**.
  - **File Naming Rules**: Use only `ASCII` characters (avoid spaces, special symbols, and non-English letters).
- **Grading Process**: The TAs will evaluate your project based on:
  - Your report write-up.
  - Your code implementation.
  - Benchmarking your code's performance.

1. Computing π with MPI [10 points]

1.1. Parellization using MPI

The parellel implementation of the π computation is based on the idea of dividing the interval $[0, 1]$ into N subintervals. Each process computes the contribution to π from its assigned subinterval and then all contributions are summed together using MPI. In listing 1 we show the implementation of the parallel π computation using MPI.

Listing 1: Parallel π computation using MPI

```
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);

h = 1.0 / (double)N;

MPI_Barrier(MPI_COMM_WORLD);
time_start = walltime();

for (uint64_t i = (uint64_t)rank; i < N; i += (uint64_t)size) {
    double x = (i + 0.5) * h;
    local_sum += 4.0 / (1.0 + x * x);
}

MPI_Reduce(&local_sum, &sum, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
double time = walltime() - time_start;
if (rank == 0) {
    pi = sum * h;
    printf("# pi N procs time [secs]\n");
    printf("%.15f %20" PRIu64 " %3d %.8f\n", pi, N, size, time);
    }

MPI_Finalize();
```

Each process starts from its rank and iterates over the indices of the subintervals assigned to it by incrementing the index by the total number of processes in each iteration. This way, each process computes the contribution to π from its assigned subintervals without overlap. After the local sums are computed, they are reduced to a single sum using `MPI_Reduce`, which is then used to compute the final value of π on the root process (rank 0).

1.2. Strong scaling

To evaluate the strong scaling of the parallel π computation, we run the program with a fixed problem size N and varying number of processes. The results are shown in figure 1, where we plot the speedup achieved by the parallel implementation compared to the serial version.

The strong scaling results obtained with MPI show a near-linear increase in speedup for a small number of processes, closely following the ideal trend. As the number of processes increases, the measured speedup progressively deviates from the ideal curve, highlighting the impact of communication overhead and reduced computational workload per process. Nevertheless, the scalability remains good, reaching a speedup close to 100 with 128 processes.

Comparing these results with the OpenMP reduction implementation in Table 1, we observe a very similar scaling behavior. Both approaches exhibit efficient parallelization at low to moderate parallelism and a loss of efficiency at higher core counts. However, OpenMP achieves slightly better efficiency at comparable core counts, likely due to the shared-memory model which avoids explicit communication overhead.

1.3. Weak scaling

The results of the weak scaling tests are shown in figure 2, where we plot the weak scaling efficiency as a function of the number of processes.

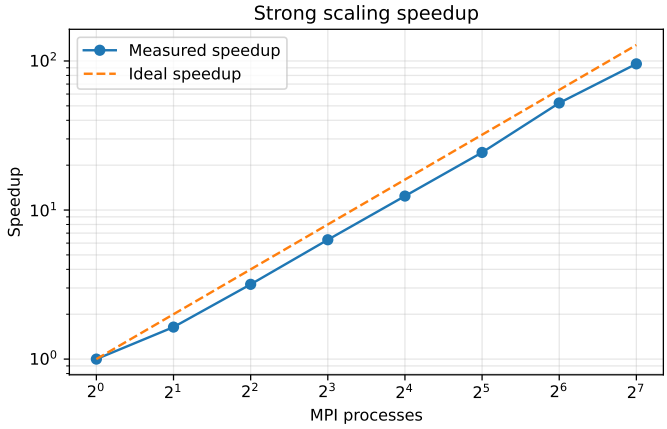


Figure 1: Strong scaling speedup of the parallel π computation. Obtained with $N = 10^9$.

Table 1: Strong scaling results using the OpenMP reduction clause ($N = 10^9$).

Threads	Speedup
1	1.00
2	1.84
4	3.26
8	6.39
16	8.86
32	17.66
64	31.06

The weak scaling efficiency remains high across all tested configurations, staying close to the ideal value of 1. This indicates that the workload is well balanced among MPI processes and that the parallelization strategy is effective.

A slight degradation in efficiency is observed as the number of processes increases, particularly at the highest core counts. This behavior is expected and can be attributed to increasing communication overhead and synchronization costs, which become more significant as more processes are involved.

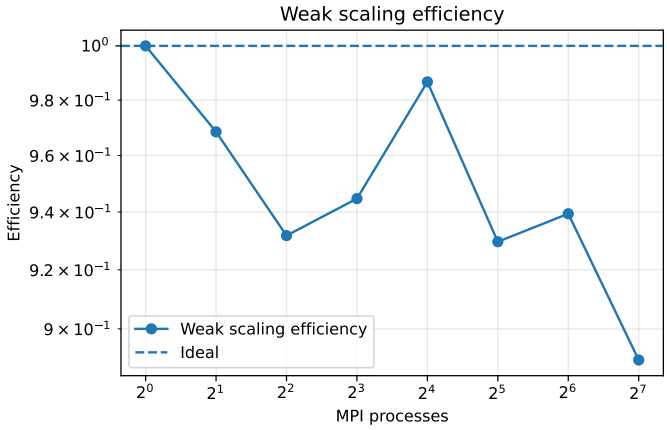


Figure 2: Weak scaling efficiency of the parallel π computation.

2. Ring sum using MPI [10 points]

In listing 2 we show the implementation of the ring sum using MPI.

Listing 2: Ring sum using MPI

```
int main(int argc, char *argv[]) {
int size, rank;
  MPI_Init(&argc, &argv);
  MPI_Comm_size(MPI_COMM_WORLD, &size);
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);

  int send_to   = (rank + 1) % size;
  int recv_from = (rank - 1 + size) % size;

  int sum = rank;
  int value = rank;
  int recv_value;

  for (int i = 0; i < size - 1; i++) {
    MPI_Sendrecv(&value, 1, MPI_INT, send_to, 0,
                 &recv_value, 1, MPI_INT, recv_from, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);

    sum += recv_value;
    value = recv_value;
  }

  printf("Process %d: Sum = %d\n", rank, sum);

  MPI_Finalize();
  return 0;
}
```

We implemented the ring sum algorithm using a one-dimensional circular topology, where each process communicates with two neighbors: the next process in increasing rank direction and the previous one, defined respectively as $(\text{rank} + 1) \bmod P$ and $(\text{rank} - 1 + P) \bmod P$, where P is the number of processes.

Each process initializes a local accumulator with its own rank and a separate variable containing the value to be propagated through the ring. At every iteration, a process sends its current value to the right neighbor and simultaneously receives a value from the left neighbor. The received value is added to the local sum and then forwarded in the next iteration. After $P - 1$ iterations, each process has received all other ranks exactly once, thus computing the global sum.

To avoid potential deadlock issues, we used the `MPI_Sendrecv` routine, which combines sending and receiving into a single operation. This ensures that all processes can safely exchange messages without the risk of circular waiting that may arise when using separate blocking `MPI_Send` and `MPI_Recv` calls. The communication pattern is therefore fully synchronized and deadlock-free.

Finally, all processes obtain the same result, confirming the correctness of the implementation as one can see from the output shown in listing 3.

Listing 3: Output of ring sum

```
Process 1: Sum = 6
Process 0: Sum = 6
Process 2: Sum = 6
Process 3: Sum = 6
```

3. Cartesian domain decomposition and ghost cells exchange [20 points]

3.1. Cartesian 2D communicator

In listing 4 we show the implementation of the Cartesian 2D communicator using MPI.

Listing 4: Cartesian 2D communicator using MPI

```
1  int rank, size, i, j, dims[2], periods[2], rank_north, rank_south,
       rank_west, rank_east;
2  double data[DOMAINSIZE*DOMAINSIZE];
3  MPI_Request request;
4  MPI_Status status;
5  MPI_Comm comm_cart;
6  MPI_Datatype data_ghost;
7
8  dims[0]= 3;
9  dims[1]= 4;
10 periods[0]= 1;
11 periods[1]= 1;
12
13 MPI_Cart_create(MPI_COMM_WORLD, 2 ,dims , periods , 0 , &comm_cart);
14
15 MPI_Cart_shift(comm_cart, 1, 1, &rank_south, &rank_north);
16 MPI_Cart_shift(comm_cart, 0, 1, &rank_west, &rank_east);
```

The code first defines the Cartesian topology of the process grid by setting `dims[0] = 3` and `dims[1] = 4`, corresponding to a 3×4 decomposition. The array `periods` is set to 1 in both directions, meaning that the topology is periodic both horizontally and vertically: processes at one boundary are connected to processes on the opposite boundary. This implements the cyclic communication pattern required by the exercise.

The call to `MPI_Cart_create` constructs a new communicator, `comm_cart`, in which all processes are arranged according to this two-dimensional Cartesian topology. Using a Cartesian communicator makes neighbor discovery much simpler than manually computing ranks.

The two calls to `MPI_Cart_shift` are then used to determine the ranks of the neighboring processes in the two coordinate directions. In particular, the first call identifies the south and north neighbors, while the second call identifies the west and east neighbors.

3.2. Derived data type for column boundary

In listing 5 we show the implementation of the derived data type used for boundary communication.

Listing 5: Derived data type for column boundary using MPI

```
1 MPI_Type_vector(SUBDOMAIN, 1, DOMAINSIZE, MPI_DOUBLE, &data_ghost);
2 MPI_Type_commit(&data_ghost);
```

The local domain is stored in a one-dimensional array, and the boundary elements that have to be exchanged are not always contiguous in memory. For this reason, a derived MPI datatype is introduced with `MPI_Type_vector`.

The datatype `data_ghost` describes `SUBDOMAIN` elements of type `MPI_DOUBLE`, separated by a stride of `DOMAINSIZE` elements in memory. Hence, it represents a strided boundary segment of the local array. This is exactly the kind of memory layout that appears in the implementation shown in listing 6, where one class of boundaries is accessed through elements that are spaced by one full leading dimension. After its definition, `MPI_Type_commit` makes the datatype available for communication.

3.3. Exchange ghost cells

In listing 6 we show the implementation of the ghost cells exchange using MPI.

Listing 6: Ghost cells exchange using MPI

```
1  /* send top interior row (j = SUBDOMAIN) to north, receive bottom ghost
       row (j = 0) from south */
2      MPI_Sendrecv(&data[SUBDOMAIN + 1 * DOMAINSIZE], 1, data_ghost,
3                   rank_north, 0,
4                   &data[0 + 1 * DOMAINSIZE], 1, data_ghost,
5                   rank_south, 0,
6                   comm_cart, MPI_STATUS_IGNORE);
7
8      /* send bottom interior row (j = 1) to south,
9         receive top ghost row (j = SUBDOMAIN+1) from north */
10     MPI_Sendrecv(&data[1 + 1 * DOMAINSIZE], 1, data_ghost,
11                  rank_south, 1,
12                  &data[(SUBDOMAIN + 1) + 1 * DOMAINSIZE], 1, data_ghost,
13                  rank_north, 1,
14                  comm_cart, MPI_STATUS_IGNORE);
15
16
17     /* send right interior column (i = SUBDOMAIN) to east,
18        receive left ghost column (i = 0) from west */
19     MPI_Sendrecv(&data[1 + SUBDOMAIN * DOMAINSIZE], SUBDOMAIN, MPI_DOUBLE,
20                  rank_east, 2,
21                  &data[1 + 0 * DOMAINSIZE], SUBDOMAIN, MPI_DOUBLE,
22                  rank_west, 2,
23                  comm_cart, MPI_STATUS_IGNORE);
24
25     /* send left interior column (i = 1) to west,
26        receive right ghost column (i = SUBDOMAIN+1) from east */
27     MPI_Sendrecv(&data[1 + 1 * DOMAINSIZE], SUBDOMAIN, MPI_DOUBLE,
28                  rank_west, 3,
29                  &data[1 + (SUBDOMAIN + 1) * DOMAINSIZE], SUBDOMAIN,
                        MPI_DOUBLE,
30                  rank_east, 3,
31                  comm_cart, MPI_STATUS_IGNORE);
```

This part of the code performs the exchange of ghost cells with the four neighboring processes of the Cartesian grid. The communication is split into four steps, one for each neighboring direction. In each step, an interior boundary of the local subdomain is sent to the corresponding neighbor, while the matching ghost boundary is received from the opposite side.

An important point is that the implementation follows the indexing convention induced by the linearized storage `data[i + j * DOMAINSIZE]`. As a consequence, the different geometric boundaries of the local domain are not all represented in memory in the same way: some are stored as strided segments and are therefore communicated through the derived datatype `data_ghost`, whereas others are contiguous in memory and can be sent directly as `SUBDOMAIN` entries of type `MPI_DOUBLE`.

For this reason, the first two `MPI_Sendrecv` calls use the derived datatype `data_ghost`. They exchange those boundaries that, under the chosen storage layout, are represented by elements separated by a stride of `DOMAINSIZE`. The last two `MPI_Sendrecv` calls instead exchange boundaries that are contiguous in memory and can therefore be communicated with the basic datatype `MPI_DOUBLE`.

Thus, the distinction in the code is not between "rows" and "columns" in an abstract geometric sense, but between strided and contiguous memory segments in the chosen array layout. With this interpretation, the implementation is fully consistent: the appropriate MPI datatype is used in each case according to how the corresponding boundary is stored in memory.

The use of `MPI_Sendrecv` is particularly important because it avoids possible deadlock issues that could arise if all processes tried to perform blocking send operations before posting matching receives. By combining send and receive into one single communication call, the ghost-cell exchange is both safe and simple to implement.

The output is shown in listing 7.
From it one can see the ranks of the neighboring processes and the data array after the ghost cell exchange, confirming that the communication was successful and that the ghost cells have been updated with the correct values from the neighbors.

Listing 7: Output of ghost cell exchange

```
North rank 6, South rank 4, West rank 1, East rank 9
data of rank 5 after communication
j
^
| -1.0  6.0  6.0  6.0  6.0  6.0  6.0 -1.0
|  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
|  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
|  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
|  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
|  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
|  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
| -1.0  4.0  4.0  4.0  4.0  4.0  4.0 -1.0
----------------------------------------> i
```

4. Parallelizing the Mandelbrot set using MPI [30 points]

4.1. Partitioning of the image

In listing 8 we can see the implementation of the function `createPartition`.

Listing 8: Partitioning of the image

```
Partition createPartition(int mpi_rank, int mpi_size) {
    Partition p;

    // TODO: determine size of the grid of MPI processes (p.nx, p.ny),
    //       see MPI_Dims_create()
    int dims[2];
    MPI_Dims_create(mpi_size , 2 , dims );
    p.ny = dims[0];
    p.nx = dims[1];

    // TODO: Create cartesian communicator (p.comm), we do not allow the
    //       reordering of ranks here, see MPI_Cart_create()
    MPI_Comm cart_comm;
    int periods[2] = {1, 1};
    MPI_Cart_create(MPI_COMM_WORLD, 2, dims, periods, 0, &cart_comm);
    p.comm = cart_comm;

    // TODO: Determine the coordinates in the Cartesian grid (p.x, p.y),
    //       see MPI_Cart_coords()
    int coords[2];
    MPI_Cart_coords(cart_comm , mpi_rank , 2 , coords);
    p.y = coords[0];
    p.x = coords[1];

    return p;
}
```

The function `createPartition` defines a two-dimensional Cartesian decomposition of the MPI processes. The use of `MPI_Dims_create` allows to automatically determine a balanced process grid $(p.ny, p.nx)$, which tends to minimize load imbalance by distributing processes as evenly as possible across both dimensions.
A Cartesian communicator is constructed through `MPI_Cart_create`. Although periodic boundary conditions are enabled in both directions, this choice is not strictly required for the Mandelbrot computation, since no inter-process communication is needed.
Each process retrieves its coordinates in the Cartesian topology using `MPI_Cart_coords`. These coordinates uniquely identify the position of the process in the logical grid and are later used to map the process to a specific portion of the global image.

In listing 9 we can see the implementation of the function `updatePartition`.

Listing 9: Updating the partitioning of the image

```
Partition updatePartition(Partition p_old, int mpi_rank) {
    Partition p;

    // copy grid dimension and the communicator
    p.ny = p_old.ny;
    p.nx = p_old.nx;
    p.comm = p_old.comm;

    // TODO: Update the coordinates in the cartesian grid (p.x, p.y) for
       given
    //       mpi_rank, see MPI_Cart_coords()
    int coords[2];
    MPI_Cart_coords(p.comm , mpi_rank , 2 , coords);
```

```
13      p.y = coords[0];
14      p.x = coords[1];
15
16      return p;
17  }
```

The function `updatePartition` recomputes the Cartesian coordinates for a given rank while preserving the communicator and the grid dimensions. This ensures consistency of the mapping without reconstructing the full topology.

4.2. Dimensions and start/end of local domain

In listing 10 we can see the implementation of the function `createDomain`.

Listing 10: Dimensions and start/end of local domain

```
1  Domain createDomain(Partition p) {
2      Domain d;
3
4      // TODO: Compute size of the local domain
5      d.nx = IMAGE_WIDTH/p.nx;
6      d.ny = IMAGE_HEIGHT/p.ny;
7
8      // TODO: Compute index of the first pixel in the local domain
9      d.startx = p.x * d.nx;
10     d.starty = p.y * d.ny;
11
12     // TODO: Compute index of the last pixel in the local domain
13     d.endx = d.startx + d.nx - 1;
14     d.endy = d.starty + d.ny - 1;
15
16     return d;
17 }
```

The function `createDomain` assigns to each MPI process a rectangular subdomain of the global image. The local domain sizes are computed as

$$\text{d.nx} = \frac{\text{IMAGE_WIDTH}}{\text{p.nx}}, \qquad \text{d.ny} = \frac{\text{IMAGE_HEIGHT}}{\text{p.ny}},$$

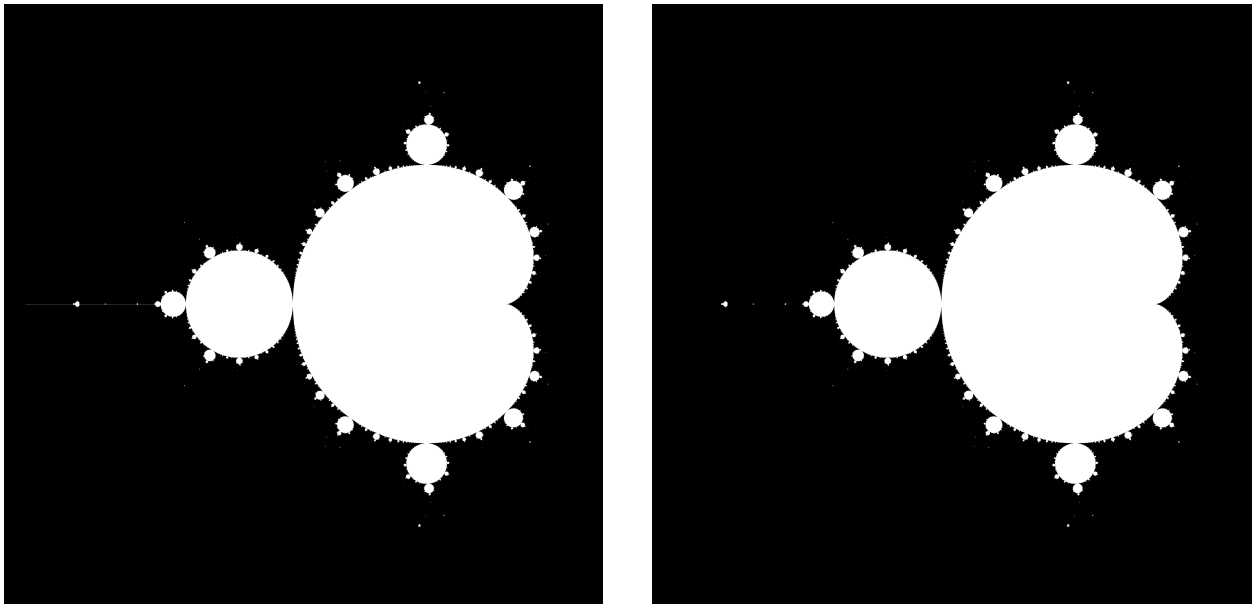
which corresponds to a block decomposition of the image. This approach is simple and efficient, as it avoids overlap between processes and leads to contiguous memory regions.
The starting indices (`d.startx, d.starty`) are determined from the Cartesian coordinates of the process, while the ending indices (`d.endx, d.endy`) define the inclusive bounds of the local subdomain.

This implementation assumes that the global image dimensions are divisible by the number of processes in each direction. While this assumption is acceptable for the current setup, a more general implementation would handle non-divisible dimensions by distributing remainder pixels.

4.3. Output check

Figure 3 compares the Mandelbrot set obtained with the parallel MPI implementation to the one produced by the serial code. The two images are visually identical, showing that the domain decomposition, the mapping from local to global pixel indices, and the final reconstruction of the global image on the master process are all correct.

No visible artifacts such as discontinuities or seams appear at subdomain boundaries in the parallel result. This confirms that each process computes its assigned region correctly and that the local domains are assembled consistently into the final image. Since the Mandelbrot computation itself is purely local, this visual agreement with the serial implementation provides a clear verification of correctness.



(a) Parallel implementation

(b) Serial implementation

Figure 3: Comparison of the Mandelbrot set computed with the parallel MPI implementation and the serial implementation.

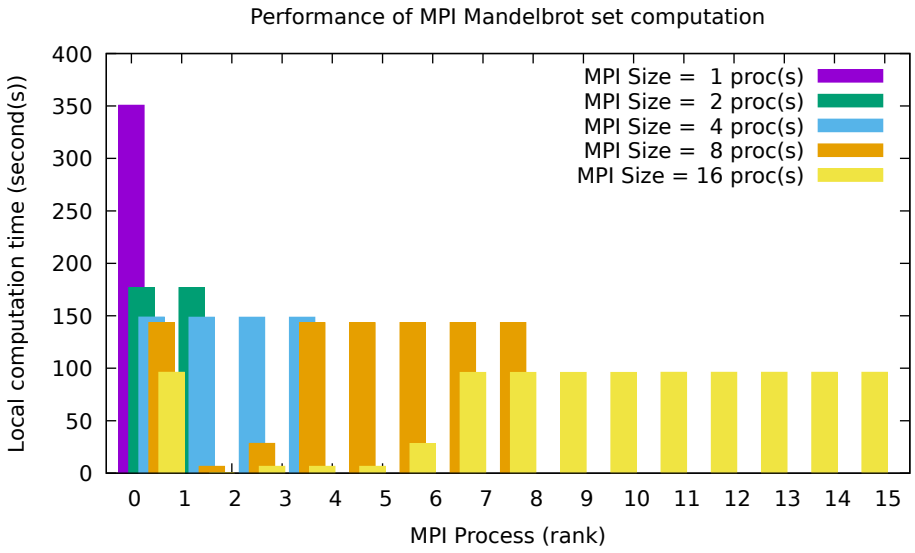


Figure 4: Performance of the parallel Mandelbrot set implementation.

4.4. Performance

Figure 4 presents the local computation time for each MPI process at different levels of parallelism.

A significant variability in execution time across processes can be observed. This is due to the intrinsic nature of the Mandelbrot set, where the number of iterations required for each pixel strongly depends on its position in the complex plane. Regions close to the boundary of the set require substantially more iterations, resulting in higher computational cost for the corresponding processes.

As a consequence, the workload is not uniformly distributed, and some processes complete their computation much earlier than others. This leads to load imbalance, which limits the overall parallel efficiency since the total execution time is determined by the slowest process.

Increasing the number of processes reduces the size of each subdomain but does not eliminate the imbalance. On the contrary, finer partitioning can make the variability more pronounced.

4.5. Possible performance improvements

Several strategies could be adopted to improve the performance of the MPI Mandelbrot implementation.

A first possible improvement concerns load balancing. The current block decomposition assigns each process one contiguous rectangular portion of the image. Since the Mandelbrot set is highly non-uniform, some subdomains contain many points close to the boundary of the set and therefore require significantly more iterations than others. This leads to processes with very different execution times. A better strategy would be to distribute the work in a more balanced way, for example by assigning rows or small tiles cyclically among the MPI processes. In this way, each process would receive a mixture of easy and expensive regions, reducing the imbalance.

Performance could also be improved by optimizing the local computation kernel. Since each process computes many independent pixels, the inner loops are good candidates for shared-memory parallelism through OpenMP, leading to a hybrid MPI+OpenMP implementation.

A further optimization would be to improve data locality and vectorization in the pixel computation loop. Careful restructuring of the computation, compiler optimization flags, and ensuring contiguous memory access patterns could help the compiler generate more efficient code.

5. Parallel matrix-vector multiplication and the power method [30 points]

We now show how all the TODOs in the provided code template have been implemented.
In listing 11 we show how we implement the partitioning of the "n" rows of the matrix evenly among the "size" MPI processes.

Listing 11: Partitioning of the matrix rows among MPI processes

```
if (rank < n % size) {
    nrows_local = n / size + 1;
    row_beg_local = rank * nrows_local;
  } else {
    nrows_local = n / size;
    row_beg_local = (n % size) * (n / size + 1)
                  + (rank - (n % size)) * (n / size);
  }

  row_end_local = row_beg_local + nrows_local - 1;
```

The code in Listing 11 defines the row-wise data distribution of the matrix across the MPI processes. Since the matrix is partitioned by rows, each process is responsible for a contiguous block of rows and therefore computes only the corresponding entries of the matrix-vector product. The distribution is made as balanced as possible: every process receives either $\lfloor n/size \rfloor$ or $\lfloor n/size \rfloor + 1$ rows, so that the load imbalance is at most one row. This is important for parallel efficiency, because the work associated with the matrix-vector multiplication is proportional to the number of rows owned by each process. The variables `row_beg_local` and `row_end_local` identify the global row range assigned to each rank, which is then used consistently throughout the power iteration to map local computations onto the correct global indices.

Then, we were asked to broadcast the random initial guess vector to all MPI processes. This was done using the `MPI_Bcast` function, as shown in listing 12.

Listing 12: Broadcasting the initial guess vector to all MPI processes

```
MPI_Bcast(y, n, MPI_DOUBLE, 0, MPI_COMM_WORLD);
```

The operation shown in Listing 12 ensures that all MPI processes start from the same initial guess vector. In the power method, every process needs access to the full current iterate, because even though each rank stores only a subset of matrix rows, the multiplication of one local row with the vector requires all vector entries. For this reason, the vector generated on rank 0 is broadcast to the entire communicator. This collective communication step establishes a consistent initial state on all processes and allows the subsequent iterations to proceed in parallel without ambiguity about the input vector.

In listing 13 we show the implementation of the power method loop.

Listing 13: Power method loop for parallel matrix-vector multiplication

```
    // Power method
  double theta, error;
  double* y_local = (double*) calloc(nrows_local, sizeof(double));
  double* v       = (double*) calloc(n, sizeof(double));
  int iter;

  MPI_Barrier(MPI_COMM_WORLD);
  double time_start = walltime();

  // Build recvcounts and displacements for MPI_Allgatherv
  int* recvcounts = (int*) calloc(size, sizeof(int));
  int* displs     = (int*) calloc(size, sizeof(int));

  for (int p = 0; p < size; ++p) {
    if (p < n % size) {
      recvcounts[p] = n / size + 1;
      displs[p] = p * recvcounts[p];
```

```
18      } else {
19        recvcounts[p] = n / size;
20        displs[p] = (n % size) * (n / size + 1)
21                  + (p - (n % size)) * (n / size);
22      }
23    }
24      for (iter = 0; iter < niter; ++iter) {
25
26      // Normalize vector: v = y / ||y||_2
27      double norm2_local = 0.0;
28      for (int i_local = 0; i_local < nrows_local; ++i_local) {
29        int i_global = row_beg_local + i_local;
30        norm2_local += y[i_global] * y[i_global];
31      }
32
33      double norm2 = 0.0;
34      MPI_Allreduce(&norm2_local, &norm2, 1, MPI_DOUBLE, MPI_SUM,
            MPI_COMM_WORLD);
35
36      double norm = sqrt(norm2);
37
38      for (int i_global = 0; i_global < n; ++i_global) {
39        v[i_global] = y[i_global] / norm;
40      }
41
42      // Matrix-vector multiply: y_local = A_local * v
43      for (int i_local = 0; i_local < nrows_local; ++i_local) {
44        y_local[i_local] = 0.0;
45        for (int j_global = 0; j_global < n; ++j_global) {
46          y_local[i_local] += A[i_local*n + j_global] * v[j_global];
47        }
48      }
49
50      // Gather full y on all processes
51      MPI_Allgatherv(y_local, nrows_local, MPI_DOUBLE,
52                     y, recvcounts, displs, MPI_DOUBLE,
53                     MPI_COMM_WORLD);
54
55      // Compute eigenvalue: theta = v^T y
56      double theta_local = 0.0;
57      for (int i_local = 0; i_local < nrows_local; ++i_local) {
58        int i_global = row_beg_local + i_local;
59        theta_local += v[i_global] * y[i_global];
60      }
61
62      MPI_Allreduce(&theta_local, &theta, 1, MPI_DOUBLE, MPI_SUM,
            MPI_COMM_WORLD);
63
64      // Check convergence: error = ||y - theta v||_2
65      double error2_local = 0.0;
66      for (int i_local = 0; i_local < nrows_local; ++i_local) {
67        int i_global = row_beg_local + i_local;
68        double r = y[i_global] - theta * v[i_global];
69        error2_local += r * r;
70      }
71
72      double error2 = 0.0;
73      MPI_Allreduce(&error2_local, &error2, 1, MPI_DOUBLE, MPI_SUM,
            MPI_COMM_WORLD);
74
```

```
75      error = sqrt(error2);
76
77      if (verbose && rank == 0) {
78        printf("iteration / theta / error: %4d / %15.5f / %25.15e\n",
79               iter, theta, error);
80      }
81
82      if (error < tol * fabs(theta)) break;
83    }
84
85    MPI_Barrier(MPI_COMM_WORLD);
86    double time_end = walltime();
```

Listing 13 contains the core parallel implementation of the power method. The algorithm combines local computation with collective MPI communication in each iteration. First, every process computes the contribution of its locally owned entries to the Euclidean norm of the current iterate, and `MPI_Allreduce` is used to combine these partial results into the global norm. This allows all ranks to normalize the vector consistently. Each process then performs the matrix-vector multiplication only for its local block of rows, producing the partial result `y_local`. Since the next power iteration again requires the full vector on every process, the local pieces are exchanged with `MPI_Allgatherv`, which is necessary because the row distribution may be nonuniform.

The same parallel pattern is used for the computation of the Rayleigh quotient and the convergence criterion. Each process evaluates its local contribution to the scalar product $v^T y$ and to the residual norm $\|y - \theta v\|_2$, and the global values are reconstructed with `MPI_Allreduce`. In this way, the expensive matrix-vector multiplication is distributed across processes, while the collective operations maintain a synchronized global view of the iterate and of the convergence information. Overall, the implementation follows a replicated-vector, row-distributed-matrix strategy: the matrix is partitioned to distribute the computational work, whereas the vector is kept available on all processes to simplify the multiplication and the iterative update.

5.1. Correctness

We first verified the correctness of our implementation. This was by running the program on all the three test cases provided in the template, which consist of matrices with known dominant eigenvalues. In all cases, the computed eigenvalue converged to the expected value within the specified tolerance, confirming that the power method is correctly implemented and that the parallelization does not introduce errors. One can see the output for the first test case in listing 14.

Listing 14: Output of the power method for the first test case

```
1     Running: p=1, test_case=1, n=16, niter=100, tol=1e-12
2 ### 1, 16, 1, 16.000000, 0.000043 ###
3
4 Running: p=2, test_case=1, n=16, niter=100, tol=1e-12
5 ### 2, 16, 1, 16.000000, 0.000062 ###
6
7 Running: p=4, test_case=1, n=16, niter=100, tol=1e-12
8 ### 4, 16, 1, 16.000000, 0.000256 ###
9
10 Running: p=1, test_case=2, n=16, niter=100, tol=1e-12
11 ### 1, 16, 0, 1.000000, 0.000036 ###
12
13 Running: p=2, test_case=2, n=16, niter=100, tol=1e-12
14 ### 2, 16, 0, 1.000000, 0.000075 ###
15
16 Running: p=4, test_case=2, n=16, niter=100, tol=1e-12
17 ### 4, 16, 0, 1.000000, 0.000159 ###
18
19 Running: p=1, test_case=3, n=16, niter=100, tol=1e-12
20 ### 1, 16, 100, 15.999987, 0.000169 ###
21
22 Running: p=2, test_case=3, n=16, niter=100, tol=1e-12
```

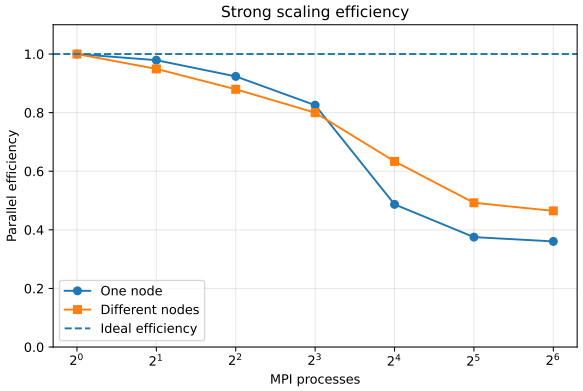
```
### 2, 16, 100, 15.999999, 0.000572 ###

Running: p=4, test_case=3, n=16, niter=100, tol=1e-12
### 4, 16, 100, 15.999972, 0.001331 ###
```

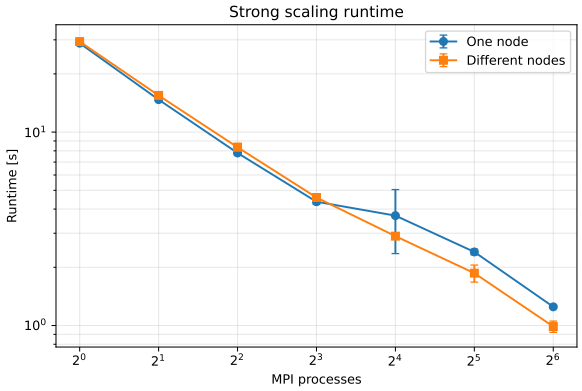
The results reported in Listing 14 confirm that the parallel implementation reproduces the expected dominant eigenvalues for all three verification cases and for all tested numbers of MPI processes. For test case 1, the dominant eigenvalue is correctly recovered as 16, while for test case 2 the method converges to the expected value 1. In test case 3, the approximation is again very close to the exact dominant eigenvalue, with only a small residual discrepancy due to the finite number of iterations and the slower convergence behavior of this case.
An important observation is that the computed eigenvalue remains essentially unchanged when increasing the number of processes from $p = 1$ to $p = 2$ and $p = 4$. This shows that the row-wise matrix distribution, the collective communications, and the global reductions are all functioning correctly.

5.2. Strong scaling

To assess the strong scaling behavior of the MPI implementation, we fixed the problem size to $n = 10\,000$, the maximum number of iterations to $niter = 300$, and the tolerance to a negative value, $\text{tol} = -10^{-6}$, so that the algorithm always performs the full number of iterations. The study was carried out for $p = 1, 2, 4, 8, 16, 32, 64$ MPI processes and for two process distributions: all MPI processes on a single node, and one MPI process per node on different nodes.



(a) Strong scaling efficiency.



(b) Strong scaling runtime.

Figure 5: Strong scaling results for the parallel power method for two MPI process distributions: all processes on one node and one process per node on different nodes.

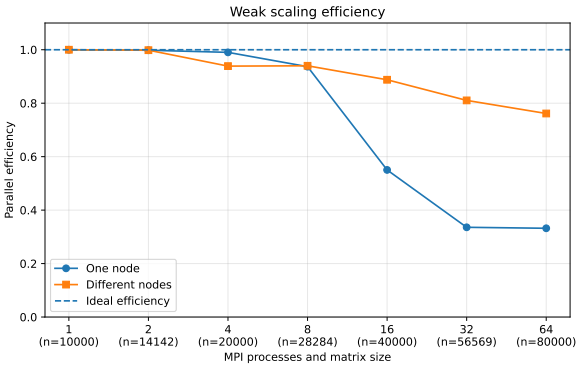
Figure 5 shows that the implementation scales well up to a moderate number of MPI processes, with efficiency close to ideal up to approximately 8 processes in both configurations. For larger process counts, the efficiency decreases significantly, which indicates that communication and synchronization overheads become increasingly important compared with the local computation. This behavior is expected, since each power-method iteration requires several collective communications, including global reductions and a global gather of the distributed vector.

Comparing the two process distributions, the multi-node configuration is slightly slower at small to intermediate process counts, reflecting the higher cost of inter-node communication. However, for the largest process counts, the different-node configuration becomes competitive and even slightly better than the one-node case. A plausible explanation is that, when all ranks are placed on one node, the execution becomes more affected by contention for shared node resources such as memory bandwidth and internal communication paths. In contrast, distributing one MPI rank per node reduces such local contention, although it introduces network communication overhead.

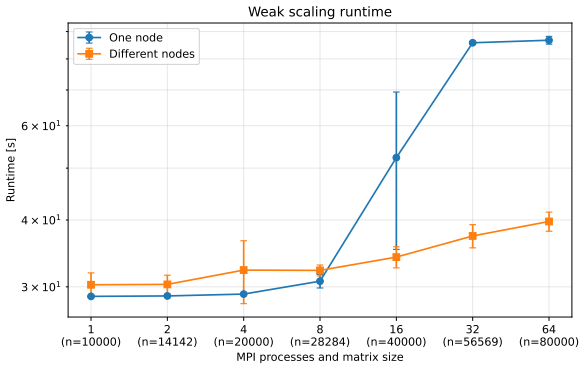
5.3. Weak scaling

For the weak scaling study, we ran the code with $p = 1, 2, 4, 8, 16, 32, 64$ MPI processes and increased the matrix size approximately proportionally to $\sqrt{p}$. Since the matrix has size $n \times n$, both the total memory requirement and the total computational work scale as n^2. Therefore, choosing $n(p) \propto \sqrt{p}$ keeps the memory

required per process and the work performed by each process approximately constant as the number of MPI processes increases. In practice, starting from $n_0 = 10\,000$ for one MPI process, we used $n(p) \approx 10\,000\sqrt{p}$. The maximum number of iterations was fixed to $niter = 300$, the tolerance was set to $\text{tol} = -10^{-6}$, and the study was repeated for two MPI process distributions: all processes on one node and one MPI process per node on different nodes.



(a) Weak scaling efficiency of the parallel power method.



(b) Weak scaling runtime of the parallel power method.

Figure 6: Weak scaling results for the parallel power method for two MPI process distributions: all processes on one node and one process per node on different nodes.

Figure 6 shows that the implementation exhibits good weak scaling up to about 8 MPI processes in both configurations, since the runtime remains nearly constant and the efficiency stays close to the ideal value. For larger numbers of processes, the one-node configuration deteriorates significantly: the runtime increases sharply from $p = 16$ onward and the efficiency drops well below one. This indicates that, although the work per process is kept approximately constant, the execution becomes increasingly limited by contention for shared node resources, such as memory bandwidth, as well as by communication and synchronization overheads.

The different-node configuration shows a more favorable trend at large process counts. Its runtime grows more gradually and the weak scaling efficiency remains substantially higher than in the one-node case for $p \geq 16$. This suggests that distributing one MPI process per node alleviates local contention effects, even though inter-node communication is more expensive.